DOCUMENTAÇÃO TÉCNICA · CÓDIGO-FONTE

101spas

Guia do código

O que cada pasta e cada arquivo faz,
e como o jogo funciona por dentro.



46	módulos em src/	**10.757**	linhas em src/*.js
200	espadas no jogo	**84**	inimigos definidos
433	receitas de criação	**54**	chefes (4 + 50)
126	efeitos de espada	**15+**	telas e painéis

REPOSITÓRIO	jk-jhon1/101spas
BRANCH	arena/565ceae5-101spas
COMMIT	651f46c
JOGO	120 → 200 espadas
GERADO EM	24/10/2026

Documento técnico gerado a partir da leitura do código-fonte. Todos os números foram medidos no repositório.

Sumário

PARTE I — VISÃO GERAL

1. O que é o 101spas 4
2. Como o projeto é montado 5
3. Mapa das pastas 5
4. A arquitetura em camadas 6
5. O laço do jogo 8
6. Os dados que alimentam o jogo 8

PARTE II — A PASTA RAIZ

7. Arquivos da raiz, um a um 12

PARTE III — A PASTA SRC/ (46 MÓDULOS)

8. Bloco 00–01 · base e dados 16
9. Bloco 02–07 · mundo, luz e arte 17
10. Bloco 08–10 · física, combate e efeitos 24
11. Bloco 11–14 · conteúdo: espadas, jogador, inimigos 27
12. Bloco 15–16 · interface, saves e controles 32

PARTE IV — A PASTA DEV/

13. Ferramentas e testes 36
14. Como rodar as verificações 39

PARTE V — A PASTA DOCS/

15. Catálogo, chefes e imagens 41

PARTE VI — COMO FUNCIONA POR DENTRO

16. O caminho de um golpe 43
17. Como uma espada é definida 44
18. Como a arte é gerada 44
19. Salvamento, mundos e arquivos 45
20. Os Tiers 7–10 (espadas 121–200) 45
21. Convenções e armadilhas do código 46

APÊNDICES

A. Todos os módulos em uma tabela 48
B. Comandos e ganchos de teste 49

PARTE I

Visão geral

O que o jogo é, como os arquivos se juntam num único HTML e quais são as camadas do código.

1. O que é o 101spas

101spas é um sandbox 2D de plataforma em pixel-art, jogado no navegador. O jogador minera, constrói, enfrenta inimigos e chefes e forja espadas: são **200 espadas** divididas em **10 tiers**, cada uma com atributos, uma mecânica única e uma arte própria. O jogo tem dois modos: **Aventura** (mundo gerado, progressão, chefes) e **Criativo** (arsenal completo, voo livre, sem morte).

Tecnicamente, é um projeto sem framework e sem dependências em tempo de execução: **um único** `index.html` de cerca de 985 KB, com todo o JavaScript, o CSS e o HTML embutidos. Não há servidor obrigatório, nem build de terceiros, nem módulos ES: o jogo é uma sequência de 43 scripts que compartilham o mesmo escopo global. Abrir o arquivo no navegador já funciona; servir por HTTP acrescenta o salvamento em IndexedDB e a API de gamepad, que exigem uma origem de verdade.

> **Em uma frase**
>
> O repositório guarda o código-fonte em pedaços legíveis (a pasta `src/`); o `build.js` costura esses pedaços em um arquivo só (`index.html`); o `serve.js` publica esse arquivo para jogar no navegador.

Números medidos no repositório

O quê	Quantidade	Onde vive
Módulos JavaScript do jogo	46 arquivos · 10.757 linhas	src/*.js
Folha de estilo e template	270 + 133 linhas	src/style.css · src/template.html
Espadas	200 (20 por tier, 10 tiers)	src/11_swords.js · src/11b_swords_x.js
Tipos de bloco e parede	62 tiles · 10 paredes	src/01_data.js
Itens e receitas	584 itens · 433 receitas	src/01_data.js · src/11c_gear_x.js
Inimigos comuns	84 definições	src/13_enemies.js
Chefes	4 clássicos + 50 novos	src/13b_bosses.js · src/13c..13g
Efeitos de espada	101 em FX + 25 em FX2	src/10_fx.js · src/11b_swords_x.js
Equipamento novo	50 minérios · 50 armaduras · 50 acessórios · 5 picaretas	src/11c_gear_x.js
Mecânicas de equipamento	50 acessórios + 50 passivas + 10 conjuntos	src/10b_gear_fx.js

2. Como o projeto é montado

O jogo não usa `import`, `require`, bundler ou transpilador. O `build.js` lê **todos** os arquivos de `src/` cujo nome começa com dígitos, ordena pelo nome e concatena o conteúdo em um único bloco de script, separado por comentários `/* ===== arquivo.js ===== */`. Depois injeta esse bloco e o CSS no `template.html` e grava o `index.html`.

```
node build.js        →   OK index.html 985 KB; 46 módulos
```

Como todos os arquivos caem no mesmo escopo, uma função declarada em `15_ui.js` pode ser chamada por `16_main.js` sem nenhum tipo de importação. O preço disso é que **a ordem importa**: constantes e objetos avaliados no topo de um arquivo precisam que as definições de que dependem já tenham rodado. É por isso que os nomes têm prefixos numéricos — eles são a ordem de carga e, ao mesmo tempo, o "endereço" de cada módulo na pilha.

1. **src/*.js**
   46 módulos numerados de 00 a 16. Cada um cuida de um assunto: dados, mundo, arte, combate, interface, laço principal.
2. **build.js**
   Ordena, concatena, escapa a sequência </script> quando ela aparece dentro de uma string e injeta o resultado no template.
3. **src/template.html + style.css**
   O esqueleto do documento (canvas, HUD, painéis, menus) e a folha de estilo que dão forma à interface em DOM.
4. **index.html**
   Arquivo único e autocontido. É o que o jogador abre. Nunca deve ser editado à mão: o próximo build sobrescreve.

> **Regra de ouro**
>
> Edite sempre `src/` e rode `node build.js` depois. Se você mexer só no `index.html`, a mudança evapora no próximo build; se mexer só no `src/`, o jogo aberto no navegador continua com a versão antiga.

Por que um arquivo só?

O jogo foi desenhado para ser portátil: dá para mandar o `index.html` por e-mail, abrir de um pendrive ou hospedar em qualquer lugar (inclusive no GitHub Pages, que só publica arquivos estáticos) sem depender de caminhos relativos, MIME types corretos ou servidor configurado. Tudo o que o jogo precisa — inclusive as imagens, que são desenhadas por código em canvas — vai dentro do arquivo.

3. Mapa das pastas

O repositório tem três pastas de trabalho e alguns arquivos soltos na raiz. Cada uma tem um papel bem definido:

Pasta	Papel	Quem usa
src/	Todo o código do jogo, em 46 módulos + CSS + template. É a única pasta que entra no build.	build.js
dev/	Scripts de verificação: validação sem navegador, testes de equipamento e de arte dos ícones, testes de navegador (Playwright) e geradores de imagens de conferência. Nada daqui entra no jogo.	desenvolvedor, npm test
docs/	Catálogo de espadas em Markdown/CSV, lista de chefes e capturas de tela usadas no README.	pessoas que leem o projeto

Arquivos da raiz

Arquivo	Para que serve
index.html	O jogo pronto (gerado). ~985 KB, autocontido.
build.js	Costura src/ em index.html. Sem dependências.
serve.js	Servidor estático mínimo (só index.html, README e docs/) para jogar por http.
package.json	Nome, scripts (`build`, `serve`, `validate`, `test`) e a única dependência de desenvolvimento (Playwright).
README.md	Documentação para quem chega: como jogar, como buildar, o catálogo e as capturas.
.gitignore / .nojekyll	Ignora capturas temporárias e node_modules; impede o GitHub Pages de filtrar as pastas.

4. A arquitetura em camadas

Apesar de tudo estar num escopo só, o código é organizado em camadas bem separadas. Uma camada só conversa com a de baixo (ou com a de cima por meio de funções de evento). Entender essa pilha é o atalho para saber **onde mexer** quando algo precisa mudar:

1

Base (00–01)

Matemática, RNG determinístico, ruído Perlin, cores, e todos os dados estáticos: tiers, tiles, itens, receitas.

2

Mundo e arte (02–07)

A matriz de blocos, o gerador procedural, a luz, os sprites de tiles, as copas, os sprites dos chefes, os ícones, as espadas e o céu.

3

Simulação (08–09b)

Estado global, colisão, partículas, dano, status, projéteis, arcos de golpe e o despacho de efeitos (hooks).

4

Efeitos e conteúdo (10–14)

Biblioteca de efeitos, catálogo de espadas, jogador, inimigos, chefes, IA e desenho de tudo que se move.

5

Interface e sistemas (15–15g)

HUD, inventário, catálogo, mapa, NPCs, salvamento em mundos, gamepad, toque e exportação de arquivo.

6

Motor (16)

Canvas em camadas, input, câmera, laço de quadro, pipeline de render e a inicialização.

Onde mexer para cada tipo de mudança

Quero…	Vá para	Camada
criar um item ou receita nova	01_data.js	Base
mudar o relevo, os biomas ou a ordem de geração	03_worldgen.js	Mundo
ajustar o dano, o crítico ou a receita de uma espada	11_swords.js · 11b_swords_x.js	Conteúdo
criar um efeito de espada novo	10_fx.js (FX) ou 11b (FX2)	Efeitos
mudar a aparência de um bloco	05_gfx.js	Arte
desenhar uma espada nova	06b_swordart.js (A2)	Arte
criar um inimigo comum	13_enemies.js (`enemy(...)`)	Conteúdo
criar um chefe	13c_bossx.js + 13d..13g + 05g..05j	Conteúdo + arte
mexer no HUD, no inventário ou no catálogo	15_ui.js	Interface
mudar o salvamento ou os cards de mundo	15c_save.js · 15e_worlds.js	Sistemas
mexer no laço, no render ou no input	16_main.js	Motor

Essa separação é o que permite, por exemplo, acrescentar um tier inteiro de 20 espadas mexendo em **três arquivos** (o catálogo novo, a arte e a tabela de tiers) e em mais nada — foi exatamente o que aconteceu com os tiers 7 a 10.

5. O laço do jogo

O jogo roda em `requestAnimationFrame`. A cada quadro, `frame(now)` calcula o delta de tempo (limitado a 50 ms, para uma aba em segundo plano não teleportar o jogador), atualiza controle e toque, e então faz duas coisas: **simula** (`step`) e **desenha** (`render`).

A simulação segue sempre a mesma ordem, e essa ordem é significativa: o jogador anda antes de os inimigos pensarem; os inimigos se movem antes de os projéteis serem atualizados; os projéteis são testados contra os inimigos **antes** de as partículas e os textos voarem. O espasmo de dano, o congelamento de quadro (`hitStop`) e a câmera vêm no fim.

Etapa de step(dt)	O que acontece
updatePlayer	Física do jogador, mineração, uso de item, ataque, i-frames.
updateNpcs / updateEnemies / bxUpdate	NPCs, inimigos comuns, chefes clássicos e novos.
updateProjs	Move projéteis, testa colisão com tiles, inimigos e jogador.
updateDrops / updateParts / updateTexts	Itens no chão, partículas e números flutuantes.
stepLiquids / stepFalling	Água e lava (em orçamento de tempo), areia e blocos sem apoio.
spawnTick	Sorteia inimigos conforme camada, bioma, noite e estágio.
autosave · câmera	Salva a cada 60 s; suaviza e limita a câmera ao mundo.

O pipeline de render (5 passadas)

O desenho acontece em **baixa resolução** e só no fim é ampliado por um fator inteiro — é isso que dá o pixel quadrado e nítido. São cinco passadas:

1. **Céu**
   Gradiente por hora do dia, estrelas, sol/lua, nuvens e colinas em paralaxe, desenhados em um canvas do tamanho da tela em “pixels do mundo”.

2. **Cena**
   Tiles com auto-tiling, copas, paredes, drops, NPCs, inimigos, chefes, jogador, projéteis, partículas, líquidos e o contorno da mineração.

3. **Luz**
   A cena é multiplicada pelo mapa de luz (propagação de opacidade por tile, RGB). O alpha original é preservado com destination-in.

4. **Brilhos**
   Tochas, núcleos, lâminas e projéteis ganham halos aditivos em resolução cheia, com blending “lighter”.

5. **Sobreposições**
   Números de dano, escurecimento, flash de tela, efeitos de tempo parado, vinheta de lava/água, retículo do controle.

6. Os dados que alimentam o jogo

Quase todo o “conteúdo” do jogo é dado, não código: os arquivos de dados declaram objetos e o resto do motor os lê. Isso vale para blocos, itens, receitas, espadas, inimigos e chefes.

As estações de criação

O campo `st` de uma receita diz **onde** ela pode ser feita. Cada estação também satisfaz as anteriores, o que evita obrigar o jogador a manter cinco bancadas na base:

Estação (`st`)	Nome no jogo	Também aceita	Onde aparece
bench	Bancada	—	Chão, feita de madeira
furnace	Fornalha	—	Fundição de minério
anvil	Bigorna	—	Ferro e aço
advanvil	Bigorna Avançada	anvil	Hardmode
forge	Forja Celeste	anvil · advanvil	Pós-jogo / tiers altos

No código, quem valida isso é `STATION_SAT` (a matriz de compatibilidade) e `STATION_NAME` (o texto exibido). `canCraft` no `12_player.js` procura um bloco-estação perto do jogador e checa se ele satisfaz o `st` da receita.

> **A regra que sustenta tudo**
>
> Se um sistema precisa de um valor fixo (dano de um tier, cor de um item, receita de uma espada), esse valor mora nos dados — nunca espalhado pelo motor. É o que permite gerar o catálogo, validar o jogo por script e mudar conteúdo sem tocar na lógica.

Dado	Estrutura	Definido em
Tiles (62)	`T.<nome>` → id; tabela `TD` com dureza, drops, som, estação, opacidade	01_data.js
Paredes (10)	`WALL/WD` — fundo que veda luz e permite moradia	01_data.js
Itens (379)	`IT[id]` com nome, tipo, tier, ícone, empilhamento, uso	01_data.js
Receitas (323)	`RECIPES[]` com saída, quantidade, ingredientes e estação	01_data.js
Espadas (200)	Objetos `SwordItemSO` com atributos, receita, arte e lista de efeitos	11_swords.js · 11b_swords_x.js
Inimigos (84)	`ED[chave]` com vida, dano, velocidade, IA (`kind`), drops	13_enemies.js
Chefes novos (50)	`BX[id]` com atributos, sigilo, condição de invocação, golpes (geradores)	13d..13g_bx_*.js

A vantagem dessa separação aparece nos testes: o `dev/validate.js` carrega apenas os módulos de dados e de lógica (sem canvas, sem DOM) e consegue conferir o catálogo inteiro em milissegundos — 200 espadas, 20 por tier, faixas de dano, nomes únicos, receitas com ingredientes existentes e estações válidas.

Um exemplo: acrescentar um item novo

Para entender como os dados conversam entre si, o caminho de um item novo é curto e mostra as três camadas ao mesmo tempo:

1. **Dado**
   Registre o item em `01_data.js` (`item(...)`): id, nome, tipo, tier, empilhamento e uso. Sem isso, nada mais o reconhece.

2. **Origem**
   Dê uma fonte ao item: uma receita em `RECIPES` (`rec(...)`) ou um drop em `13_enemies.js` — id, quantidade mínima/máxima e chance.

3. **Imagem**
   Se ele não for espada, `06_icons` o desenha a partir do tipo e da cor; se for, entra num catálogo de espadas.

4. **Interface**
   Só isso: inventário, receitas, lojas e catálogo leem a mesma tabela — o item aparece sozinho, com tooltip e preço.

5. **Verificação**
   `node dev/validate.js` acusa se algum ingrediente não existe ou se a estação é inválida; `dev/reach.js` acusa item sem fonte.

PARTE II

A pasta raiz

Os arquivos soltos que fazem o projeto existir: o jogo compilado, o build, o servidor e a documentação.

7. Arquivos da raiz, um a um

`index.html`

GERADO 9090 linhas

O jogo inteiro em um arquivo — não editar à mão

Documento HTML autocontido, de cerca de 985 KB, que carrega o template, o CSS e os 46 módulos concatenados em um único `<script>`.

É o artefato que o jogador abre. O `build.js` o reescreve a partir de `src/`; qualquer edição manual é perdida no próximo build. Ele existe para que o jogo funcione sem servidor, sem rede e sem caminhos relativos.

- Estrutura: `<canvas id="view">` (o jogo), canvas do fundo do menu, HUD em DOM, painéis e menus, e um bloco `<script>` com tudo.
- A string `</script>` dentro do código é escapada pelo build para não fechar o bloco antes da hora.
- Abrir por `file://` funciona, mas o salvamento em IndexedDB e o gamepad ficam limitados; por http tudo funciona.

USA nenhum (autocontido)
USADO POR jogador · GitHub Pages · serve.js

`build.js`

FERRAMENTA 31 linhas

Concatena src/ em index.html

Script Node sem dependências que junta os módulos, o CSS e o template num arquivo só.

Lê os nomes de `src/`, filtra os que começam com dígitos, ordena alfabeticamente (o que equivale à ordem numérica), concatena com marcadores de arquivo e injeta no template. Normaliza quebras de linha e escapa `</script>`.

- A ordenação alfabética é a ordem de carga: `10_fx.js` vem antes de `11_swords.js`, e assim por diante.
- O CSS entra no marcador `/*__STYLE__*/`; o JS no `/*__SCRIPT__*/`.
- Imprime o tamanho final e a contagem de módulos — um jeito rápido de saber se algum arquivo novo entrou.

read() · replaceAll() · (script principal)

USA fs, path
USADO POR npm run build · fluxo de desenvolvimento

`serve.js`

FERRAMENTA 95 linhas

Servidor estático mínimo

Servidor HTTP de ~90 linhas que publica apenas `index.html`, `README.md` e a pasta `docs/`.

Serve para jogar por uma origem de verdade (o que libera IndexedDB e Gamepad API) e para testar no celular pela rede local. Não expõe `src/`, `dev/` nem `node_modules/`.

- Lista branca de caminhos + `path.posix.normalize` e checagem de `realpath` para barrar `..` e links simbólicos.
- Cabeçalho `Cache-Control: no-cache`: recarregar sempre traz a versão nova.
- Porta opcional por argumento: `node serve.js 9000`.

resolve() · (handler do http)

USA http, fs, path
USADO POR npm run serve · testes de navegador

`package.json`

CONFIG 30 linhas

Scripts e dependência de desenvolvimento

Declara nome, scripts de build/servidor/teste e a única dependência de desenvolvimento (Playwright).

O jogo em si não tem dependências: nada é instalado para jogar. O Playwright só é necessário para os testes que abrem um Chromium de verdade (`npm install && npx playwright install chromium`).

- `npm run build` → `node build.js`.
- `npm test` → validação rápida sem navegador (validate, reach, seeds, layouts, transfer).
- `npm run test:browser` → a bateria completa com Chromium.

USA playwright (dev)
USADO POR npm

`README.md`

DOCUMENTAÇÃO 700 linhas

A porta de entrada do projeto

Explica o jogo, como jogar, como buildar, o catálogo de espadas, os chefes e as capturas de tela.

É o documento que uma pessoa lê ao chegar no repositório: controles, modos, progressão, estrutura de pastas e as imagens de `docs/img`. Também registra as decisões de projeto (por que um arquivo único, por que sem dependências).

USA docs/img/*.png
USADO POR quem visita o repositório

Do arquivo aberto ao primeiro quadro

Abrir o `index.html` dispara uma sequência curta, mas que passa por quase todos os módulos. Vale conhecê-la porque é o primeiro lugar onde um erro de carregamento aparece:

1. **O navegador lê o HTML**

   Monta o `canvas`, a HUD e os painéis; o CSS dá forma a tudo isso. Nenhum script rodou ainda.

2. **O bloco <script> roda de cima para baixo**

   Cada módulo declara suas funções e constantes no escopo global, na ordem dos prefixos.

3. **O último módulo chama a inicialização**

   `16_main` liga os eventos, prepara o canvas e mostra o menu do título — o `Worlds` lê o IndexedDB para listar os mundos.

4. **O jogador escolhe ou cria um mundo**

   `genWorld` roda em fatias com barra de progresso e monta a matriz de blocos, os baús, os NPCs e o ponto de partida.

5. **O laço começa**

   `requestAnimationFrame` chama `frame`, que simula (`step`) e desenha (`render`) até o jogador sair.

O que acontece se você mexer em cada arquivo

Arquivo	Mexer nele serve para…	Cuidado
index.html	nada — ele é descartável	Qualquer edição some no próximo build.
build.js	mudar o empacotamento (ex.: minificar)	É a única peça que conhece a ordem de carga dos módulos.
serve.js	publicar mais arquivos (ex.: uma pasta de assets)	A lista branca é o que impede servir `src/` e `dev/`.
package.json	adicionar scripts de verificação	A dependência de desenvolvimento não deve vazar para o jogo.
README.md	documentar o projeto para quem chega	Nada no jogo lê este arquivo — é só documentação.

Atalhos de leitura do código

Para quem vai abrir o projeto pela primeira vez, estes são os pontos de entrada mais úteis — o lugar onde cada assunto começa:

Se você quer entender…	Comece por	Depois veja
como um mundo é gerado	`03_worldgen.js` (genWorld)	`02_world.js`, `16_main.js`
como a luz funciona	`04_light.js` (compute/toImage)	`16_main.js` (render, 3ª passada)
como um sprite é feito	`05_gfx.js` → `05b_pixel.js`	`06_icons.js`, `06b_swordart.js`, `06c_gearart.js`
como o dano é calculado	`09_combat.js` (hitEnemy)	`09b_swing.js`, `10_fx.js`
como uma espada vira efeito	`11b_swords_x.js` (FX2)	`10_fx.js` (FX)
como o save é gravado	`15c_save.js` (Worlds)	`15f_transfer.js`
como a interface é montada	`15_ui.js` (renderArsenal)	`src/template.html`, `style.css`

> **Se a tela ficar em branco**
>
> Quase sempre é um erro de JavaScript na carga: abra o console do navegador. Um `ReferenceError` na primeira linha de um módulo interrompe a leitura do bloco inteiro — e nada depois dele é definido. É por isso que os testes de navegador falham citando o arquivo e a linha.

PARTE III

A pasta src/

Os 46 módulos do jogo, na ordem em que são carregados: dados, mundo, arte, combate, conteúdo, interface e motor.

8. Bloco 00–01 · base e dados

Os dois primeiros módulos não desenham nada e não simulam nada: eles fornecem ferramentas matemáticas e **todo o conteúdo estático** do jogo. São os únicos que os testes carregam sempre, porque tudo depende deles.

src/00_util.js

BASE 84 linhas

Matemática, sorteio determinístico e cores

Funções curtas usadas por todos os outros módulos: interpolação, ângulos, ruído, hash, conversão de cor e escape de HTML.

Concentra o “vocabulário” numérico do jogo. O ponto mais importante é o `mulberry32`: um gerador pseudoaleatório **determinístico**, que permite que uma semente produza sempre o mesmo mundo. O `Perlin` gera o relevo e as nuvens; `hash2` faz variações estáveis por coordenada (sem guardar nada em memória).

- `clamp`, `lerp`, `dist`, `approach`, `angDiff` — utilidades de movimento e ângulo.
- `mulberry32(seed)` e `strSeed` — mundos reproduzíveis; `Perlin` com `n2` e `fbm` para relevo em camadas.
- `hex2rgb`, `rgbs`, `mix`, `shade`, `tint` — toda a paleta do jogo sai daqui; `escHtml` protege nomes digitados pelo jogador.

clamp · lerp · rnd · rndi · pick · chance · dist · approach · angDiff · mulberry32 · Perlin · hash2 · hex2rgb · shade · escHtml

USA nada
USADO POR todos os módulos

src/01_data.js

DADOS 259 linhas

Tiers, tiles, paredes, itens e receitas

A “planilha” do jogo: 11 tiers de raridade, 62 blocos, 10 paredes, 584 itens e 433 receitas de criação (os minérios, armaduras e acessórios novos ficam em `11c_gear_x.js`).

Declara os dados por meio de pequenas funções de registro (`tile`, `item`, `rec`, `ORE`) e monta as tabelas que o resto do motor consulta: `T` (id dos tiles), `TD` (dureza, drop, som, estação), `IT` (itens), `RECIPES` (receitas) e `STATION_NAME`/`STATION_SAT` (estações e o que cada uma libera).

- `TIERS` guarda nome, cor e faixa de multiplicador de dano de cada tier — é o que colore slots, tooltips e o catálogo.
- Cada tile declara dureza, opacidade (para a luz), se é sólido, se cai (areia) e o que dropa.
- As receitas ligam item final → ingredientes + estação (bancada, fornalha, bigorna, bigorna avançada, forja).

tile() · wallItem() · item() · rec() · ORE() · T · TD · IT · RECIPES

USA 00_util
USADO POR todo o resto do jogo

9. Bloco 02–07 · mundo, luz e arte

Este é o maior bloco em número de arquivos. Ele responde por duas coisas: **como o mundo existe** (matriz de blocos, geração procedural, luz) e **como o mundo é desenhado** (tiles, copas, chefes, ícones, espadas, céu). Nada aqui conhece o jogador; são sistemas de dados e de pintura.

`src/02_world.js`

MUNDO 230 linhas

A matriz de blocos e suas regras

Classe `World`: arrays tipados para blocos, paredes, líquidos e bioma, mais líquidos, quedas, árvores e portas.

O mundo é uma grade de 1.400 × 480 tiles de 16 px (22.400 × 7.680 px). Para não gastar memória, cada propriedade é um `TypedArray`: blocos em `Uint8Array`, paredes, quantidade e tipo de líquido, "já revelado no mapa" e a exposição ao céu. A classe oferece leitura/escrita com limites, atualização de suporte, derrubada de árvore inteira e a simulação de fluidos em orçamento.

- `get/set`, `solid`, `wallAt`, `liqAt` — acesso seguro (fora do mundo devolve rocha-mãe).
- `stepLiquids(budget)` — água e lava escorrem apenas nas células "acordadas" (`liqAct`), em lotes por quadro; água + lava = obsidiana.
- `stepFalling`, `checkSupport`, `fellTree`, `toggleDoor` — areia cai, blocos sem apoio desabam, árvore cai inteira.

class World · stepLiquids · stepFalling · set · setWall · fellTree · checkSupport · doorColumn · layerAt

USA 00_util, 01_data
USADO POR 03_worldgen, 08_phys, 12_player, 16_main

`src/03_worldgen.js`

MUNDO 439 linhas

Gerador procedural em fatias

Gera o mundo inteiro em 11 etapas, cedendo o controle entre elas para a barra de carregamento não travar a página.

`genWorld` é um **gerador** (`function*`): cada `yield` devolve `[progresso, mensagem]` e o `startGame` consome um pouco por quadro. A semente define o layout dos biomas, o estilo do relevo, as cavernas, os minérios, as árvores, as ilhas do céu e o inferno. `chooseSeed` sorteia sementes comparando o "layout" das já usadas para garantir mundos visivelmente diferentes.

- Etapas: biomas e relevo → lagos → cavernas → minérios → areia/argila → árvores e plantas → ilhas flutuantes → inferno → baús e estruturas → detalhes → finalização.
- `makeLayout(seed)` sorteia ordem e largura dos 4 biomas de terra, o tamanho dos oceanos e o ponto de partida (sempre na floresta).
- `spawnHardmodeOres` espalha cobalto..adamantita quando o Guardião do Abismo é derrotado.

genWorld* · makeLayout · layoutRuns · chooseSeed · lootForDepth · spawnHardmodeOres · Perlin

USA 00_util, 01_data, 02_world
USADO POR 16_main (startGame), dev/seeds.js, dev/layouts.js

src/04_light.js

MUNDO 106 linhas

Iluminação dinâmica

Classe `Lighting`: propaga luz RGB célula a célula usando a opacidade do tile, em duas iterações de quatro varreduras.

Implementa o item “iluminação dinâmica” do GDD. A luz do céu e as fontes (tochas, lava, projéteis, núcleos) entram numa grade do tamanho da tela; a propagação faz quatro varreduras (esquerda, direita, cima, baixo) duas vezes, atenuando conforme a opacidade da célula de origem. O resultado vira um `ImageData` usado como camada multiplicativa.

- `compute(world, x0, y0, w, h, sky)` — monta a grade de luz da área visível.
- `toImage(img, ambiente)` — escreve a luz no buffer que o render usa.
- `skyLight(time)` — clareia e escurece conforme o ciclo de dia/noite.

class Lighting · compute · toImage · sample · skyLight

USA 00_util, 02_world
USADO POR 16_main (render), 13_enemies (spawn)

src/05_gfx.js

ARTE 346 linhas

Sprites 16×16 dos blocos

Pinta cada tipo de bloco proceduralmente num buffer RGBA e devolve sprites prontos, com auto-tiling por máscara e variantes.

Não existe nenhum arquivo de imagem no jogo: todos os tiles são desenhados por código. O módulo tem um mini “pincel” (`bSet`, `bOver`, `bRect`, `bAlpha`, `bClr`), funções de pintura por material (tijolo, tronco, minério, folha) e um atlas que empacota sprites para desenhar rápido. `tileMask` calcula a máscara de vizinhança para emendar blocos (grama no topo, contornos) e `getCracks` produz os estágios de rachadura da mineração.

- Cada tipo de tile gera 1 sprite + variantes; a escolha da variante é estável por coordenada.
- Máscara de 4 bits → sprite certo para bordas, cantos e interior.
- Galhos, rachaduras e paredes de fundo também são gerados aqui.

genTile · getTileSprite · getWallSprite · tileMask · getCracks · getBranchSprite · atlasAlloc

USA 00_util, 01_data
USADO POR 16_main (drawWorldTiles), 05c_canopy

src/05b_pixel.js

ARTE 156 linhas

Pixelização e bonecos humanos

Converte desenho vetorial em pixel-art (binariza o alfa, adiciona contorno e sombreamento de cima-esquerda) e monta os bonecos humanos.

É a ponte entre “desenhar com formas suaves” e “pixel-art de verdade”. `drawPixelized` desenha o vetor pequeno, binariza o alfa e contorna; `pixelShade` aplica luz de cima-esquerda com poucas cores. Também gera cabeça, tronco, pernas e braço do jogador, NPCs e zumbis, além dos corações da HUD.

- `pixelizeCanvas(canvas, { th, outline, shade })` — usado pelos ícones e pelas espadas antigas.
- `humanLegs / humanUpper / humanArm` — peças separadas para animar andar e golpear.
- Nada aqui toca o DOM ao carregar (o validador do Node carrega este arquivo).

pixelizeCanvas · pixelShade · drawPixelized · humanUpper · humanLegs · humanArm · heartSprites

USA 00_util, 05_gfx
USADO POR 06_icons, 12_player, 14_draw_enemies, 15_ui

src/05c_canopy.js

ARTE 164 linhas

Copas das árvores por vizinhança

Desenha a copa como um contorno arredondado calculado a partir dos 5×5 blocos ao redor, em vez de quadrados de folha.

A colisão e a geração continuam usando os mesmos tiles de folha; só o **desenho** muda. Cada bloco recebe o pedaço de copa que cai nele, então os pedaços se emendam sem costura e a copa se refaz sozinha quando uma folha é quebrada. Há uma textura que se repete a cada 256 px, três paletas (floresta, selva, pinheiro) e desenho em lote.

- `canopyOccAt` consulta vizinhança; `canopyBlur` suaviza o contorno; `genCanopySprite` pinta e cacheia.
- O lote (`drawCanopyBatch`) desenha todas as copas de uma vez, no fim da passada de tiles.

genCanopySprite · drawCanopyBatch · canopyOccAt · canopyBlur · canopyTex

USA 00_util, 01_data, 05_gfx
USADO POR 16_main (drawWorldTiles)

src/05d_bosspaint.js

ARTE 151 linhas

Ferramentas de pintura dos chefes

Buffer RGBA com rampa por pixel: polígonos, elipses, curvas, união de formas, preenchimento, contorno, halo e empacotamento em canvas.

Os chefes não são vetores pixelizados: são **pintados pixel a pixel**, com luz de cima-esquerda, poucos tons por material e um brilho de cor por chefe. Este módulo fornece o “pincel” dessa pintura. Cada pixel guarda também a rampa e o tom, o que permite clarear/escurecer depois e fazer degradês coerentes.

- `bpNew/bpSet/bpFill/bpEdge/bpGlow/bpStamp` — primitivas de pintura e contorno.
- `bpPoly/bpEll/bpSeg/bpCurve/bpUnion` — formas; `BP_BAYER` faz o pontilhado do halo.
- `bpPack` converte o buffer em canvas — é o que o desenho do jogo consome.

bpNew · bpPoly · bpEll · bpCurve · bpUnion · bpFill · bpEdge · bpGlow · bpPack

USA 00_util
USADO POR 05e_bossart, 05f_bxart, 06b_swordart

src/05e_bossart.js

ARTE 293 linhas

Sprites dos 4 chefes clássicos

Pinta Rei Slime, Olho Colossal, Guardião do Abismo e Colosso Estelar, com peças móveis separadas (íris, lanterna, anéis, órbitas).

Cada chefe tem rampas próprias e um brilho de uma cor. O sprite inteiro é gerado por “quadro” de animação e guardado em cache; as partes que se mexem são desenhadas por cima, no módulo de desenho, para poderem acompanhar o jogador ou girar.

- `bossSpr(kind, frame, fase)` devolve o canvas do corpo; `bossPart` devolve peças soltas.
- `bossWarm` pré-aquece os quadros mais usados para não engasgar no primeiro segundo da luta.

slimeSprite · eyeSprite · eyeIris · guardSprite · guardLantern · colossusSprite · bossSpr · bossPart · bossWarm

USA 00_util, 05d_bosspaint
USADO POR 14_draw_enemies

src/05f_bxart.js

ARTE 68 linhas

Infraestrutura da arte dos 50 chefes novos

Registro `bxArt`, ajudas de pintura em coordenadas relativas ao centro, rampas de corpo (`RP`) e cache de sprites.

Define o “kit” usado pelos quatro arquivos seguintes. Cada chefe se declara com tamanho, centro de colisão, número de quadros e uma função de pintura. As ajudas (`h.P`, `h.E`, `h.S`, `h.C`, `h.F`, `h.D`, `h.G`, `h.eye`, `h.star`) desenham em coordenadas relativas ao centro da caixa de colisão, o que mantém o desenho alinhado independentemente do tamanho.

- `bxRamp` gera 6 tons escuro→claro; `bxGlowRamp` gera 7 tons para o brilho com núcleo quase branco.
- `bxSpr(id, frame, variante)` pinta e cacheia; `bxWarmAll` pré-aquece todos.

bxArt · bxH · bxSpr · bxWarmAll · bxRamp · bxGlowRamp · RP

USA 00_util, 05d_bosspaint
USADO POR 05g..05j_bxart*, 14b_draw_bx

src/05g_bxart1.js

ARTE 142 linhas

Chefes novos do Ato I (8)

Pintura dos 8 primeiros chefes novos — do Despertar, com dificuldade de início de jogo.

Cada `bxArt(...)` descreve um chefe: caixa, centro, quadros e a função que pinta usando o kit de 05f. São desenhos independentes, sem reúso de silhueta, o que garante que dois chefes nunca se confundam na tela.

- Um bloco por chefe, na ordem do ato.
- Animações curtas (2–4 quadros) feitas por deslocamento de partes, não por redesenho de poses.

USA 05f_bxart, 05d_bosspaint
USADO POR 14b_draw_bx (via BOSSDRAW.bx)

src/05h_bxart2.js

ARTE 243 linhas

Chefes novos do Ato II (14)

Pintura dos 14 chefes do Pós-Olho — cristais, golems, enxames e conjuradores.

Mesmo esquema de 05g, com formas maiores e mais camadas (cristais, núcleos, asas, correntes). O brilho de cada um usa a cor do elemento que ele representa.

USA 05f_bxart, 05d_bosspaint
USADO POR 14b_draw_bx

src/05i_bxart3.js

ARTE 236 linhas

Chefes novos do Ato III (16)

Pintura dos 16 chefes do Hardmode — ceifadores, aberrações e construtos.

A partir daqui os chefes têm silhuetas mais altas e detalhes emissivos (lâminas espectrais, olhos múltiplos, runas flutuantes).

USA 05f_bxart, 05d_bosspaint
USADO POR 14b_draw_bx

src/05j_bxart4.js

ARTE 179 linhas

Chefes novos do Ato IV (12)

Pintura dos 12 chefes do Além do Colosso — os mais altos e os mais ornamentados.

Usa as mesmas ajudas, com mais pixels emissivos, halos partidos e elementos flutuantes para passar a ideia de “fim de jogo”.

USA 05f_bxart, 05d_bosspaint
USADO POR 14b_draw_bx

src/06_icons.js

ARTE 243 linhas

Ícones e sprites das 120 primeiras espadas

Desenha os ícones 32×32 de todos os itens e gera as espadas 1–120 por perfil de lâmina vetorial, pixelizado no fim.

Há duas responsabilidades: (1) o desenho de cada item na interface, por especificações declarativas (`it.ic`) — poção, minério, machado, armadura, sigilo etc.; (2) o sprite da espada no mundo, gerado a partir de `look` (formato, cor, comprimento) por `bladeProf`, que devolve meia-largura e curvatura ao longo da lâmina. O sprite é pixelizado para manter o estilo.

- `getSwordSprite` decide entre o desenho novo (pixel a pixel, de 06b) e o legado (vetorial).
- `iconCanvas(id)` monta 32×32, centraliza a espada em diagonal e escala pelo tamanho real do sprite.
- `iconURL` guarda o data-URL e as classes CSS são registradas uma vez — a interface nunca redesenha o mesmo ícone.

getSwordSprite · getSwordSpriteLegacy · bladeProf · iconCanvas · iconURL · drawIconSpec · glowDot · mk

USA 00_util, 01_data, 05b_pixel, 06b_swordart
USADO POR 12_player, 14_draw_enemies, 15_ui, 16_main

src/06b_swordart.js

ARTE 649 linhas

Arte pixel a pixel das espadas 121–200

Interpreta a receita `look.art2` e pinta cada uma das 80 espadas novas num buffer RGBA, com bisel, contorno e pixels emissivos.

O módulo mais recente e o maior arquivo de arte do projeto. Em vez de vetores, cada espada é **pintada pixel a pixel** em tamanho real: silhueta (lâmina + guarda + cabo + pomo + extras), bisel de dois tons (gume claro em cima, dorso escuro embaixo), contorno escuro de 1 px, gemas e runas com rampa própria. Trinta famílias de lâmina × dezesseis guardas × sete cabos × cinco pomos e extras dão o repertório; cada receita combina elementos diferentes, então nenhuma das 80 repete a de outra.

- `a2paint` pinta por coluna com bisel contínuo; `a2tip` desenha a ponta de cada família; `a2sep` afasta a cor da guarda da cor da lâmina quando elas são parecidas.
- `a2run` faz o bisel padrão e o `rim` luminoso das lâminas escuras; `a2em` marca pixels emissivos que brilham no render aditivo.
- `art2Sprite` devolve `{ c, w, h, ox, oy, len, sw, sh }` — `sw/sh` são a caixa real da espada, usada para escalar o ícone sem cortar.

art2Sprite · art2Paint · a2paint · a2tip · a2sep · a2run · a2em · a2outline · a2gripBase

USA 00_util, 05d_bosspaint, 08_phys
USADO POR 06_icons (ícones), 16_main (mundo)

src/06c_gearart.js

ARTE 760 linhas

Arte dos 50 minérios, 50 barras, 50 armaduras, 50 acessórios e 5 picaretas

Desenha cada peça de equipamento a partir da **semente do item**: o formato do minério, a massa e a inclinação da barra, o corte da couraça, o tipo de acessório e a cabeça da picareta mudam de peça para peça.

A ideia é que nenhum item pareça o outro. Em vez de uma silhueta por família, cada desenho recebe `(forma, tier, seed)` e decide com a semente: quantas agulhas o cristal tem e para onde se inclinam, que casca o geodo ganha, qual dos cinco cortes de couraça a armadura usa, quantos dedos a manopla tem, o comprimento do bico da bota, quantas pontas a coroa leva. Por cima entram o brasão da família (dez desenhos), o motivo do peito (tabardo, gemas, corrente, gola de pelo, ombro assimétrico, faixa rúnica), os cravos da borda e as marcas de tier. A paleta é derivada da cor do minério, com um ajuste de luminância (`gaLift`) para os metais quase pretos continuarem visíveis no fundo escuro do inventário.

- `gaRnd(seed)` é um gerador determinístico: o mesmo item desenha igual em qualquer sessão e em qualquer navegador.
- `drawOreArt` tem três formas (agulhas, geodo, lasca); a ganga é tingida pela cor do minério.
- `drawBarArt` varia a massa, a inclinação, o entalhe e a lasca; o selo da família e as marcas de tier ficam no topo.
- `drawArmorArt` combina cinco cortes × cinco degraus de escala e um motivo diferente por peça, com o brasão por cima.
- `drawAccArt` cobre treze formas (anel, pingente, manopla, botas, manto, coroa, fole, nadadeira, bolha, ampulheta, coração, escudo, vazio). Dentro das quatro maiores, **cada peça tem um desenho nomeado**: dez anéis (solitário, sinete, aro duplo, trançado, alado, garra, serpente, face, aro aberto, pedras em órbita), nove pingentes (gota, medalhão, placa, aro aberto, torque, duplo, olho, frasco, chave), dez manoplas (luva, pesada, garra, braceleto, espinhos, mão aberta, avambraço, munhequeira, placas, esfera) e dez botas (botina, cano alto, alada, greva, bico longo, gola de pelo, joelheira, sandália, salto, cravos). O índice vem de `GACC_VAR`, no catálogo — não da semente.
- `dev/test_gear_art.js` desenha os 205 ícones e compara o hash do alfa: se duas peças tiverem a mesma silhueta, o teste falha.

gaRnd · gaGem · gaSpark · gaDots · gaEdgeMarks · gaLift · drawOreArt · drawBarArt · drawArmorArt · drawArmorMotif · drawAccArt · drawPickArt · drawFamilyCrest

USA 00_util
USADO POR 06_icons (drawIconSpec), 11c_gear_x (sementes das peças)

src/07_sky.js

ARTE 100 linhas

Céu, sol, lua, nuvens e colinas

Desenha o céu em faixas de cor que mudam com a hora do dia, com estrelas, astros com halo, nuvens de pixels e colinas em paralaxe.

`drawSky` recebe o tempo normalizado (0–1 em 420 s de dia) e interpola as cores de uma tabela de sete paradas: madrugada, amanhecer, dia, tarde, pôr do sol e noite. Nuvens e estrelas têm posições estáveis por semente; as colinas usam a cor do bioma e se movem mais devagar que a câmera.

- `initSky(seed)` sorteia estrelas e nuvens; os sprites de sol/lua são gerados uma vez.
- `skyGradAt(frac)` interpola a paleta do céu por interpolação linear entre paradas.

initSky · drawSky · skyGradAt · sunSprite · moonSprite · cloudSprite · discSprite

USA 00_util, 02_world
USADO POR 16_main (render, menu)

De onde vem cada pixel

Uma dúvida comum ao abrir o projeto é “onde está a imagem do inimigo?”. Não está em lugar nenhum: cada elemento visual tem um módulo gerador. Esta tabela é o mapa.

O que aparece na tela	Quem desenha	Técnica
Blocos, paredes, rachaduras	05_gfx	Pintura por material + máscara de vizinhança
Copas das árvores	05c_canopy	Contorno calculado na vizinhança 5×5
Jogador, NPCs, zumbis	05b_pixel + 12_player	Vetor pixelizado, peças separadas
Inimigos comuns	14_draw_enemies	Formas simples + tinta de status
4 chefes clássicos	05e_bossart + 14_draw_enemies	Pintura pixel a pixel + camadas móveis
50 chefes novos	05g..05j + 14b_draw_bx	Pintura pixel a pixel, uma silhueta por chefe
Espadas 1–120 (ícones)	06_icons	Vetor pixelizado a partir de `look`
Espadas 121–200 (ícones e mundo)	06b_swordart	Pintura pixel a pixel por família
Minérios, barras, armaduras, acessórios, picaretas	06c_gearart	Um desenho nomeado por peça + brasão da família
Efeitos, projéteis, partículas	10_fx + 09_combat	Formas em canvas + brilho aditivo
Céu, astros, nuvens, colinas	07_sky	Faixas interpoladas + paralaxe

Por que gerar em vez de guardar imagens

Um único sprite de 16×16 em PNG ocuparia mais bytes que o código que desenha todos os blocos. Além disso, a paleta, o contorno e o sombreamento ficam consistentes entre tudo — e mudar uma cor de tier, por exemplo, vale para o ícone, o mundo e o catálogo ao mesmo tempo.

10. Bloco 08–10 · física, combate e efeitos

Aqui começa a simulação. O estado global `G` vive em `08_phys.js`; a colisão e as partículas também. O combate fica em `09_combat.js` (dano, status, projéteis) e o arco do golpe em `09b_swing.js`. Os efeitos das espadas são uma biblioteca à parte (`10_fx.js`) que o catálogo só referencia pelo nome.

src/08_phys.js

SIMULAÇÃO 145 linhas

Estado global, colisão, partículas e drops

Declara o objeto `G` (o estado de tudo) e as funções de colisão AABB contra tiles, além de partículas, textos flutuantes e itens no chão.

`G` é o "coração" compartilhado: estado da tela, mundo, jogador, inimigos, projéteis, câmera, luz, opções de sandbox, contadores. Este módulo também contém o movimento de corpos com colisão por eixo, consultas de líquido, altura do chão/teto, raios contra tiles, partículas (`addPart`, `burst`, `ringFx`, `tileBits`), textos que sobem e o ciclo de vida dos itens dropados.

- `bodyMove` move em passos pequenos, resolve X e Y separadamente e devolve o que foi tocado.
- `overlapSolid` e `solidPx` convertem pixels em índices de tile para testar colisão.
- `dropItem` + `updateDrops` fazem o item cair, quicar e ser recolhido com atraso.

G · bodyMove · overlapSolid · solidPx · groundY · ceilY · rayTiles · los · addPart · burst · ringFx · addText · dropItem

USA 00_util, 01_data, 02_world, 04_light
USADO POR todos os módulos de jogo

src/08b_audio.js

SIMULAÇÃO 60 linhas

Efeitos sonoros sintetizados

Sintetiza todos os sons com osciladores e ruído da WebAudio API — nenhum arquivo de áudio no projeto.

O objeto `Snd` cria um grafo uma única vez (compressor + ganho mestre + buffer de ruído de 1 s) e `sfx(nome)` dispara combinações de tom e ruído com envelope. Há um controle de repetição por nome para não saturar em quadros consecutivos.

- `Snd.init()` é chamado no primeiro gesto do jogador (política de autoplay dos navegadores).
- `Snd.play(nome)` monta o som; `mute`/`setVol` controlam o volume geral.

Snd.init · tone · noise · play · mute · setVol · sfx

USA nada
USADO POR combate, interface, NPCs, chefes

src/09_combat.js

SIMULAÇÃO 334 linhas

Dano, status, projéteis e explosões

O pipeline completo de combate: aplicar dano, calcular crítico e defesa, status, morte, projéteis de todos os formatos, feixes e explosões.

É o módulo que decide tudo o que acontece quando algo acerta algo. `hitEnemy` calcula dano (base × multiplicador × crítico − defesa), aplica elemento e status, dispara os ganchos da espada e devolve o resultado. `killEnemy` dá drops, moedas e conta chefes. `spawnProj`/`updateProjs`/`drawProj` movem e desenham projéteis de ~18 formatos diferentes, com homing, balística, perfuração e recarga por alvo.

- `dpsPush` mantém um registro de dano dos últimos 10 s, usado pelos testes para provar que a espada causa dano.
- O teto de 700 projéteis descarta os mais antigos — por isso o laço de atualização precisa tolerar buracos na lista.
- `fxCall`/`fxAll` despacham os ganchos (`onSwing`, `onApex`, `onHit`, `onProjHit`, `tick`, `stats`…) para os efeitos da espada equipada.

hitEnemy · killEnemy · dealRaw · hurtPlayer · healPlayer · applyStatus · spawnProj · updateProjs · drawProj · fireBeam · explode · fxCall · fxAll

USA 00_util, 08_phys, 10_fx, 14_draw_enemies
USADO POR 09b_swing, 10_fx, 11b, 12_player, 13_enemies, 13c_bossx

src/09b_swing.js

SIMULAÇÃO 95 linhas

O arco do golpe

Implementa o golpe como um arco (ArcSwing), estocada (Thrust) e feixe (Beam), com ápice, colisão da lâmina e clones “fantasma”.

O GDD pede que o golpe varra um arco em torno do ombro, na direção do cursor, e que os projéteis saiam na direção do arco no seu ponto médio. `startSwing` cria uma **instância** de golpe (com tempo de vida, lado, alcance e multiplicador de dano); `updateSwings` calcula o ângulo, a ponta, o rastro e chama o gancho de ápice; `swingHit` testa o segmento da lâmina contra a caixa de cada inimigo.

- Cada instância tem seu próprio conjunto de inimigos já atingidos — um clone não pode acertar o mesmo alvo duas vezes.
- Instâncias com `ghost: true` são os ecos e espelhos criados por efeitos: desenham com transparência e não aplicam “hit stop”.
- O ângulo usa aceleração e desaceleração (ease in-out) para o golpe parecer ter peso.

startSwing · updateSwings · swingHit · instAngle · instTipDist · drawSwingTrail · atkSpeedMult

USA 08_phys, 09_combat
USADO POR 12_player, 10_fx, 11b_swords_x

src/10_fx.js

EFEITOS 276 linhas

Biblioteca de efeitos das espadas

Cento e um efeitos prontos, declarados como fábricas de ganchos, mais as ferramentas que eles usam para atirar, criar áreas, invocar meteoros e ferir a tela.

O catálogo de espadas não escreve lógica de efeito: ele pede `FX.nome(...)`. Cada efeito devolve um objeto com os ganchos que quiser — `onSwing`, `onApex`, `onHit`, `onKill`, `tick`, `mod`, `stats`, `atkMult`, `init`. Isso mantém o catálogo legível e permite que o mesmo efeito seja combinado em várias espadas com parâmetros diferentes.

- `shoot`/`mkProj` criam projéteis já posicionados no ápice do golpe; `aura`, `portalAt`, `meteorFall`, `chainFx` cobrem padrões comuns.
- `screenDamage` e `breakHeldSword` implementam efeitos de escala de tela e a espada que se quebra.
- `sword(P)` devolve a espada equipada; `dmgOf(inst, mult)` calcula o dano de um efeito a partir do golpe que o originou.

FX (101 efeitos) · shoot · mkProj · aura · portalAt · homingBlade · meteorFall · chainFx · screenDamage · dmgOf · sword

USA 00_util, 08_phys, 09_combat
USADO POR 11_swords (catálogo 1–120)

11. Bloco 11–14 · conteúdo: espadas, jogador, inimigos

O bloco de conteúdo define **o que existe no jogo**: as espadas (com atributos, arte e efeitos), o jogador (inventário, criação, movimento), os inimigos comuns, os chefes clássicos e os 50 novos, além do desenho de tudo o que se move.

src/11_swords.js

CONTEÚDO 157 linhas

Catálogo das 120 primeiras espadas

A fábrica `S(...)` monta o objeto `SwordItemSO` de cada espada: atributos, arte, efeitos, receita e os métodos OnSwing/OnHitEnemy.

Cada chamada `S(num, nome, dano, velocidade, knockback, look, efeitos, texto, receita)` cria uma espada com id (`s001`...`s120`), tier calculado por `ceil(num/20)`, crítico padrão do tier e receita associada. Os textos dos tiers 1, 3 e 5 vêm do GDD; os tiers 2, 4 e 6 completam o catálogo seguindo as regras de cada tier.

- `DEF_CRIT` e `TIER_STATION` definem crítico e estação padrão por tier (tiers 7–10 estendidos aqui).
- Os métodos `OnSwing`/`OnHitEnemy` existem para cumprir o contrato do schema e delegam aos ganchos.
- O resultado entra em `SWORDS[]` e em `SWORD_BY_NUM{}`, e a receita é registrada em `RECIPES`.

S() · LK() · DEF_CRIT · TIER_STATION · SWORDS[]

USA 01_data, 10_fx
USADO POR 12_player, 15_ui, 09_combat

src/11b_swords_x.js

CONTEÚDO 425 linhas

Tiers 7–10: as 80 espadas novas (121–200)

Segunda metade do catálogo, com 20 espadas por tier, arte exclusiva (receita `look.art2`) e 25 efeitos novos (FX2).

Repete a estrutura de 11_swords com duas fábricas: `S2` (espada) e `A2` (receita de arte para o pintor de 06b). Os efeitos ficam em `FX2`, um objeto com 25 mecânicas pensadas para o pós-jogo: eco do golpe, leque de lâminas, nova, cometa, varredura, corrente, marca que detona, domínio, gravidade, escudo, dobra, divisão, almas, execução, explosão de sangue, convergência, ciclo, fenda, espelho, tempestade, parada no tempo, linha de condenação, orbes, singularidade e big bang.

- Cada espada tem nome, dano, velocidade, knockback, texto de mecânica, receita e uma arte que não repete a de outra.
- Os ganchos que um eco dispara **não** geram novos ecos (guarda contra recursão) — sem isso, os acertos cresciam exponencialmente.
- Tiers 7–8 usam a Bigorna Avançada; 9–10 usam a Forja Celeste, com Fragmentos Celestes, Núcleos Míticos e essências.

S2() · A2() · FX2 (25 efeitos) · TIER7_ST

USA 11_swords, 10_fx, 06b_swordart
USADO POR 12_player, 15_ui, 09_combat

src/10b_gear_fx.js

EQUIPAMENTO 498 linhas

Mecânicas dos acessórios, armaduras e conjuntos

As 50 passivas de armadura (`ARM_FX`), as 50 mecânicas de acessório (`ACC_FX`) e os 10 bônus de conjunto (`SET_FX`), escritas como fábricas de ganchos.

Cada peça devolve um objeto de ganchos que o motor chama em momentos fixos do jogo: `stats` (ao recalcular atributos), `tick` (a cada quadro), `mod` (antes do dano), `onHit`, `onKill`, `onHurt`, `incoming` (escolhe esquiva e reduções), `onLethal` (renascimento), `onJump`, `onLand`, `onWater`, `onPotion`, `onSwing` e `atkMult`. `rebuildGear` monta a lista ativa quando o equipamento muda e conta as famílias para ativar os conjuntos.

- Um efeito nunca se chama de novo em reação a si mesmo: dano causado por acessório entra com `gear: 1`, e os ganchos ignoram esse marcador (sem isso os efeitos se multiplicariam sem fim).
- Cada acessório tem o seu próprio estado (`P.gearState[id]`), então recargas e contadores não vazam entre peças.
- Conjuntos contam armadura + acessórios da mesma família: 2 peças dão um bônus pequeno, 4 peças o bônus completo.

ACC_FX (50) · ARM_FX (50) · SET_FX (10) · rebuildGear · gearCall · gearAtkMult · gHit · gBoom · gProj

USA 01_data, 09_combat, 10_fx
USADO POR 12_player, 09_combat, 09b_swing, 15_ui

src/11c_gear_x.js

CONTEÚDO 256 linhas

50 minérios, 50 armaduras, 50 acessórios e 5 picaretas

Dez famílias (Braseiro, Geada, Corisco, Maré, Raiz, Véu, Alvorada, Relojoaria, Vazio e Origem), cada uma com 5 minérios, 5 armaduras e 5 acessórios.

Os 50 minérios viram 50 telhados de minério no mundo (gerados por `gearPlaceOres`, chamada pelo gerador em `03_worldgen.js`) e 50 barras na Fornalha. As armaduras são forjadas com as barras da própria família, em Bigorna, Bigorna Avançada ou Forja Celeste. Os acessórios vêm de 45 chefes (um exclusivo por chefe, incluindo os 4 clássicos) e 5 são forjados.

- A dureza dos minérios vai de 2 a 14 e usa as 5 picaretas novas (poder 10 a 14), forjadas na Forja Celeste.
- Os minérios são distribuídos por profundidade: quanto mais fundo o tier, menor a oferta perto da superfície e maior no Inferno.
- `gearSourceText` mostra no tooltip de onde vem cada peça: nome do chefe, estação ou dureza da picareta.

GFAM · GORES · GARMS · GACCS · GPICKS · BX_ACC · gearPlaceOres · gearSourceText

USA 01_data, 10b_gear_fx, 13c_bossx
USADO POR 03_worldgen, 13_enemies, 13c_bossx, 15_ui

src/12_player.js

CONTEÚDO 360 linhas

Jogador, inventário e criação

O corpo do jogador (12×26 px), os 40 espaços de inventário, os equipamentos (1 armadura + 4 acessórios), as estatísticas, a criação, a mineração, a construção e o desenho.

É o módulo mais "jogável" do projeto. Cuida do movimento com colisão, pulo, natação, i-frames, morte e renascimento; do inventário de 40 posições + armadura + três acessórios; da criação de itens com verificação de estação próxima; de quebrar e colocar blocos com alcance; de usar consumíveis; e de desenhar o boneco com as peças de 05b.

- `recalcStats` soma os bônus de armadura e acessórios e chama o gancho `stats` da espada equipada.
- `canCraft`/`doCraft` checam estação (`nearStations`) e ingredientes; `missingStation` alimenta a dica na interface.
- `revealMap` revela o minimapa em raio; `killPlayer`/`respawnPlayer` tratam a morte sem apagar o mundo.

newPlayer · invAdd/invRemove/invCount · canCraft · doCraft · recalcStats · placeTile · breakTile · useHeldItem · updatePlayer · drawPlayer · revealMap

USA 01_data, 05b_pixel, 08_phys, 09_combat, 09b_swing, 15b_npc
USADO POR 16_main

src/13_enemies.js

CONTEÚDO 334 linhas

Inimigos comuns, IA e spawn

Define os 84 inimigos (com variantes elementais), oito tipos de IA, as tabelas de spawn por camada/bioma/noite/estágio e os drops.

A função `enemy(chave, dados)` registra um inimigo em `ED` com vida, dano, defesa, velocidade, massa, cor, sangue, drops e moedas. A IA é escolhida por `kind`: slime (pulos), walker (anda e vira), archer (atira), flyer (voo direto), hover (paira), ghost (atravessa paredes), swim e critter (inofensivo). `spawnTick` sorteia o que aparece conforme onde o jogador está.

- Variantes elementais reaproveitam o mesmo corpo com cor e efeito de status diferentes.
- `rollDrops` e `bossLoot` distribuem itens; `bossDefeated` marca as bandeiras de progresso.
- `findSpawnPos` procura um lugar livre e iluminado o suficiente para nascer.

enemy() · spawnEnemy · updateEnemies · aiSlime/aiWalker/aiArcher/aiFlyer/aiHover/aiGhost/aiSwim · spawnTick · rollDrops

USA 01_data, 08_phys, 09_combat
USADO POR 14_draw_enemies, 16_main, 12_player

src/13b_bosses.js

CONTEÚDO 78 linhas

Os 4 chefes clássicos

Invocação, fases e ataques do Rei Slime, do Olho Colossal, do Guardião do Abismo e do Colosso Estelar.

Cada chefe é um objeto com vida alta, defesa, padrões de ataque e um "gatilho" de invocação (item, horário ou local). `bossAI` escolhe ataques conforme a fase (percentual de vida) e `radialShots`/`bossProj` criam os projéteis temáticos.

- `summonBoss` coloca o chefe no mundo com uma posição livre (`freeSpot`).
- A morte marca a bandeira em `G.flags` e libera o próximo estágio (inclui os minérios de Hardmode).

summonBoss · bossAI · bossProj · radialShots · freeSpot

USA 13_enemies, 09_combat, 05e_bossart
USADO POR 16_main, 14_draw_enemies

src/13c_bossx.js

CONTEÚDO 306 linhas

Motor dos 50 chefes novos

Registro `BX`, condições de invocação, sigilos, repertório de golpes escritos como geradores e um kit de peças reutilizáveis (zonas, feixes, projéteis, lacaios, teleporte).

Cada chefe novo se declara com `bx(id, { ... })`: atributos, ato, condição (bioma, item, horário), mecânica única descrita em texto, e um conjunto de golpes escritos como `function*` — `yield 0.5` espera meio segundo e `yield` sozinho espera um quadro. O motor executa esses geradores, cuida das três fases, das zonas de perigo, dos projéteis telegrafados, dos lacaios e dos debuffs no jogador.

- Peças: `bxZone` (lentidão, veneno, vento, explosão com aviso), `bxBeam`, `bxShot`, `bxFan`, `bxRing`, `bxRain`, `bxCol`, `bxTele`, `bxBoom`, `bxMin`, `bxBuff`.
- `bxUpdate` roda os golpes, `bxDraw` desenha e `bxOverlay` aplica escurecimentos e avisos de tela.
- A derrota dá loot, conta para `bxDefeatedCount` e libera os próximos atos.

bx() · summonBossX · bxUpdate · bxDraw · bxZone · bxBeam · bxFan · bxRing · bxRain · bxMinionAI · bxLoco

USA 09_combat, 08_phys, 05f_bxart, 13_enemies
USADO POR 13d..13g_bx_*, 14b_draw_bx, 16_main

src/13d_bx_a1.js

CONTEÚDO 120 linhas

Chefes novos do Ato I (8)

Dados e golpes dos oito primeiros chefes novos, do Despertar.

Cada bloco declara um chefe com nome, tamanho, vida, dano, moedas, espada que dropa, receita do sigilo, condição de invocação, mecânica e os golpes. A dificuldade acompanha o começo de jogo.

USA 13c_bossx
USADO POR 13c (motor), 14b (desenho)

src/13e_bx_a2.js

CONTEÚDO 269 linhas

Chefes novos do Ato II (14)

Os catorze chefes do Pós-Olho, com mecânicas de cristal, reflexo e invocação.

Aumenta o repertório: golpes com aviso no chão, refração de raios, lacaios que precisam ser limpos e fases que mudam o cenário.

USA 13c_bossx
USADO POR 13c, 14b

src/13f_bx_a3.js

CONTEÚDO 263 linhas

Chefes novos do Ato III (16)

Os dezesseis chefes do Hardmode, com telegrafias longas e padrões densos.

Mistura zonas de perigo, feixes que varrem a arena, chuva de projéteis e teleportes em sequência. É o ato que introduz os golpes de “marca e detonação”.

USA 13c_bossx
USADO POR 13c, 14b

src/13g_bx_a4.js

CONTEÚDO 187 linhas

Chefes novos do Ato IV (12)

Os doze chefes finais, acima do Colosso Estelar, com mecânicas que alteram regras do jogo.

Inclui inversão de gravidade, sangramento acumulativo, telas divididas por fendas e execuções — os combates mais longos do jogo.

USA 13c_bossx
USADO POR 13c, 14b

src/14_draw_enemies.js

DESENHO 236 linhas

Desenho de inimigos e chefes

Desenha cada tipo de inimigo proceduralmente, aplica tinta de status e organiza as camadas dos chefes (corpo, partes, sombra, brilho).

Os inimigos comuns são desenhados por formas simples com a cor do indivíduo (que muda com status: queimando, congelado, envenenado, amaldiçoado…). Os chefes clássicos usam o sprite pintado de 05e mais camadas móveis. O registro `BOSSDRAW` permite que módulos posteriores acrescentem desenhos sem tocar neste arquivo.

- `STATUS_TINT` mistura a cor base com a cor do efeito ativo.
- `bossBegin/bossLayer/bossShadow/bossBlit` compõem o chefe em etapas, para o brilho ficar por cima do corpo.

drawEnemy · drawEnemyGlow · bossBegin · bossLayer · bossBlit · bossShadow · ec

USA 05b_pixel, 05e_bossart, 08_phys, 14b_draw_bx
USADO POR 16_main

src/14b_draw_bx.js

DESENHO 36 linhas

Desenho dos 50 chefes novos

Liga o sprite pintado de cada chefe novo ao motor de desenho, com sombra no chão e camadas móveis opcionais.

É um arquivo curto de propósito: só registra `BOSSDRAW.bx` (corpo + camadas `under`/`over`) e `BOSSDRAW.bxm` (lacaios). Toda a arte está em 05f–05j; toda a lógica em 13c. Assim, acrescentar um chefe novo não exige mexer no desenho nem no motor.

BOSSDRAW.bx · BOSSDRAW.bxm · bxGroundShadow

USA 05f_bxart, 14_draw_enemies
USADO POR 16_main (bxDraw)

12. Bloco 15–16 · interface, saves e controles

O último bloco liga o jogo ao navegador: interface em DOM, NPCs e moradia, biblioteca de mundos salvos, gamepad, toque, exportação de arquivo e o laço principal com o pipeline de render.

src/15_ui.js

INTERFACE 319 linhas

HUD, inventário, catálogo, mapa e ajuda

Toda a interface em DOM: corações, barra de itens, painéis de inventário e criação, catálogo das espadas, tooltips, minimapa, mapa grande e a tela de ajuda.

A interface não é desenhada em canvas: são elementos HTML posicionados sobre o jogo, o que dá texto nítido, rolagem e acessibilidade de graça. Este módulo monta os slots, sincroniza o inventário com o DOM, calcula o HTML de cada tooltip, filtra e ordena receitas, e renderiza o catálogo com abas.

- Os ícones viram classes CSS com `data:` URL — registradas uma vez, reutilizadas para sempre.
- `ARS_TABS` define as abas do catálogo: Tier 1 a Tier 10, “Todas”, itens, inimigos e mundo. No modo aventura só aparecem as abas de tier e “Todas”.
- `openPanel`/`closePanels` coordenam qual painel está aberto e bloqueiam o jogo enquanto isso.

initUI · buildSlotsUI · refreshInvUI · paintSlot · tipHTML · renderCrafting · renderArsenal · refreshHUD · drawMinimap · drawBigMap · helpHTML · toast

USA 01_data, 12_player, 06_icons, 15b_npc
USADO POR 16_main

src/15b_npc.js

INTERFACE 145 linhas

Moradia e NPCs

Valida salas para moradia, faz os NPCs aparecerem, atualiza seu comportamento e abre as lojas de compra e venda.

Para um NPC se mudar, a sala precisa ser fechada, ter paredes de fundo em toda a área, luz, mesa ou bancada, cadeira e ao menos uma porta. `scanRooms` faz uma busca em largura para achar salas válidas; `spawnNpcQuiet` coloca o morador; `openNpc` monta o painel de diálogo e a loja.

- `itemValue`/`sellPrice` definem preços (as espadas têm tabela por tier, estendida até o tier 10).
- `updateNpcs` mantém o NPC na sala, olhando para o jogador e falando dicas contextuais.

scanRooms · spawnNpc · updateNpcs · openNpc · itemValue · sellPrice · NPC_DEF · SHOPS

USA 01_data, 12_player, 15_ui
USADO POR 16_main, 15c_save

src/15c_save.js

SISTEMAS 241 linhas

Biblioteca de mundos salvos

Guarda vários mundos em IndexedDB, comprime os blocos com RLE, gera miniaturas, salva em fila e migra o save antigo.

O objeto `Worlds` é uma pequena biblioteca: metadados leves na loja `worlds` (nome, semente, modo, progresso, miniatura, aparelho de origem), o mundo em si na loja `data` e todas as sementes já sorteadas na loja `seeds` — para que nenhuma semente se repita nem depois de excluir um mundo. Sem IndexedDB (iframe restrito, modo privado), os mundos ficam em memória durante a sessão, com aviso.

- `snapshotGame` serializa; `rleEnc`/`rleDec` comprimem os milhões de blocos; `renderThumb` desenha a miniatura.
- Os salvamentos entram numa fila (`this.q`) para nunca haver dois ao mesmo tempo; o autosave roda a cada 60 s.
- `allocate` sorteia uma semente inédita evitando layouts parecidos com os já usados.

Worlds.allocate/save/load/rename/remove · snapshotGame · applyGame · rleEnc/rleDec · renderThumb · worldNameFromSeed

USA 02_world, 12_player, 15f_transfer
USADO POR 15e_worlds, 16_main

src/15d_gamepad.js

CONTROLES 204 linhas

Suporte a controle

Traduz o controle para o jogo: mira analógica com cursor virtual, navegação de menus e retículo na tela.

O analógico direito move um cursor virtual cuja distância do jogador define o alcance da mira; o esquerdo anda. Nos painéis, o controle emite os mesmos eventos de mouse que a interface já entende, então nenhum painel precisou ser reescrito. Se não houver controle (ou a API estiver bloqueada), o módulo fica inerte.

- `padUpdate(dt)` é chamado a cada quadro pelo laço principal, em um `try/catch` próprio.
- `padCursor/padHover/padClick` reproduzem o comportamento do mouse sobre o DOM.

padUpdate · padStick · padCursor · padClick · padPanel · padPlay · drawPadReticle

USA 12_player, 15_ui
USADO POR 16_main

src/15e_worlds.js

INTERFACE 179 linhas

Menu do título e tela “Meus Mundos”

Monta o menu inicial, a lista de mundos com cartões, criar/abrir/sair, renomear, excluir e o selo de salvamento.

É a camada de apresentação do sistema de mundos. Mostra o cartão de cada mundo (nome, bioma, modo, tempo jogado, miniatura), permite renomear e excluir, e liga os botões “Nova Aventura”, “Modo Criativo” e “Meus Mundos”. Também cuida do autosave ao sair.

- Não há campo de semente: o botão sorteia uma semente inédita e mostra a geração.
- `openWorlds` e `quitToMenu` alternam entre jogo e menu sem recarregar a página.

initWorldsUI · updateTitleMenu · renderWorlds · openWorlds · newWorld · openWorld · quitToMenu · showSaveBadge

USA 15c_save, 15f_transfer
USADO POR 16_main (initGame)

src/15f_transfer.js

SISTEMAS 261 linhas

Este aparelho, proteção e mapa em arquivo

Gera um código de aparelho, pede armazenamento persistente ao navegador e exporta/importa mundos como arquivos `.json` com CRC-32.

O código do aparelho é aleatório e guardado localmente: serve para dizer de onde um mapa veio, sem usar IP (que o navegador esconde e que muda de rede para rede). A exportação monta um arquivo autocontido com verificação de integridade; a importação **não confia em nada** do arquivo: valida o CRC e reconstrói o mundo apenas com campos e valores permitidos.

- `crc32` + `sanitizeSave` são a fronteira de segurança: um arquivo adulterado é rejeitado ou limpo.
- `Device.init()`/`Protect.autoAsk()` cuidam do código e do pedido de persistência.
- `buildWorldFile`/`parseWorldFile` convertem entre mundo e arquivo.

Device · Protect · crc32 · sanitizeSave · buildWorldFile · parseWorldFile · worldFileName · cleanSlot

USA 15c_save
USADO POR 15e_worlds, 15c_save

src/15g_touch.js

CONTROLES 80 linhas

Controles de toque

Direcional virtual, botão de ação com mira por arrasto, pulo, cura rápida, zoom e pausa para celular e tablet.

Imita o esquema de um jogo de plataforma mobile: o polegar esquerdo anda (e empurrar para cima pula), o direito ataca/minera e, ao arrastar, define a direção do golpe. Liga sozinho em telas de toque e pode ser forçado no menu de pausa.

- `touchWanted` decide se o modo de toque entra em ação.
- Os botões emitem os mesmos eventos de teclado/mouse que o resto do jogo já escuta.

initTouch · touchUpdate · touchWanted · quickHeal · touchInteract

USA 12_player, 15_ui
USADO POR 16_main

src/16_main.js

MOTOR 270 linhas

Loop, render, input e câmera

O ponto de entrada do jogo: cria os canvas em camadas, trata teclado/mouse, roda o laço de quadro, desenha as cinco passadas e inicializa tudo.

É o único módulo que conhece o DOM do jogo diretamente e o único que dispara a simulação. Ele mantém quatro canvas (cena em baixa resolução, luz, céu e a vista final), aplica o zoom inteiro (1 px do mundo = N px de tela) e executa, a cada quadro, `padUpdate` → `touchUpdate` → `step` → `refreshHUD` → `render`. Também cuida da tela de carregamento (consumindo o gerador do mundo) e do fundo animado do menu.

- `step(dt)` atualiza jogador, NPCs, inimigos, chefes, projéteis, drops, partículas, líquidos, spawn, autosave e câmera — nesta ordem.
- `render()` desenha céu, cena, luz, brilhos e sobreposições; tudo com `imageSmoothingEnabled = false` na ampliação.
- No fim, expõe ganchos (`window.__G`, `__step`, `__start`, `__SW`…) que os testes usam para dirigir o jogo.

initGame · frame · step · render · drawWorldTiles · applyZoom · setPause · startGame · finishLoad · mouseWorld

USA todos os módulos anteriores
USADO POR index.html

PARTE IV

A pasta dev/

Validação, testes de navegador e ferramentas de conferência visual. Nada daqui entra no jogo — é a rede de segurança de quem mexe no código.

13. Ferramentas e testes

O projeto tem duas famílias de verificação. A primeira roda **em Node, sem navegador**, carregando os módulos do jogo em um contexto isolado com um DOM falso: é rápida (milissegundos) e serve para validar dados e lógica pura. A segunda abre um **Chromium de verdade** com Playwright e simula um jogador: clica, aperta teclas, conecta um controle, gira o celular — e verifica o que aconteceu no jogo.

Validação e dados (sem navegador)

`dev/validate.js`

TESTE 141 linhas

Valida o catálogo inteiro

Carrega os módulos de dados no Node e confere 200 espadas, 20 por tier, faixas de dano, nomes únicos, receitas, estações e os 50 chefes.

É o teste mais rápido e o mais abrangente sobre conteúdo. Ele falha se faltar uma espada, se um tier tiver 19 ou 21, se o dano sair da faixa, se um ingrediente não existir, se a estação for inválida ou se um nome repetir. Também gera o catálogo em Markdown/CSV — mas só quando pedido, para não sujar o repositório a cada execução.

- `node dev/validate.js` → imprime um resumo por tier e `CATÁLOGO OK` no fim.
- `CATALOGO=1 node dev/validate.js` → além de validar, reescreve `docs/catalogo_*.md/.csv`.

USA fs, path, stubs de DOM
USADO POR npm test

`dev/reach.js`

TESTE 34 linhas

Tudo é obtível?

Verifica se cada item usado em alguma receita tem fonte (bloco, drop, baú, loja ou outra receita) e se as 200 espadas são craftáveis.

Responde à pergunta “dá para terminar o jogo?”. Percorre o grafo de receitas para trás e aponta ingredientes sem origem e itens inalcançáveis. Hoje: 200/200 espadas craftáveis, nenhum ingrediente sem fonte.

USA mesmos stubs de validate
USADO POR npm test

`dev/seeds.js · dev/layouts.js`

TESTE 110 linhas

Variedade dos mundos

Amostram milhares de sementes para provar que os mundos variam e que a política de sorteio nunca repete um layout parecido.

`seeds.js` mede a distribuição dos layouts; `layouts.js` compara mundos completos lado a lado (ordem de biomas, largura, oceanos, relevo) e valida a regra de “semente inédita”.

USA 03_worldgen
USADO POR npm test

dev/transfer.js

TESTE 233 linhas

Arquivo de mundo

Testa o formato `.json` de exportação sem navegador: CRC, ida e volta exata, arquivos adulterados, fuzz e a lógica de importação.

É o teste de segurança do recurso de exportar/importar. Modifica arquivos válidos bit a bit, refaz o CRC para tentar burlar a checagem, injeta campos proibidos e confirma que a importação recusa ou limpa tudo o que não deveria entrar.

USA 15f_transfer
USADO POR npm test

dev/test_gear_art.js

TESTE 126 linhas

Nenhuma peça de equipamento repete outra

Desenha os 205 ícones novos (50 minérios, 50 barras, 50 armaduras, 50 acessórios e 5 picaretas) em um canvas de verdade e compara a **silhueta** de cada um com todas as outras.

A regra “cada peça tem aparência própria” precisa de um teste objetivo, porque arte gerada por código repete com facilidade. O script carrega `src/` num contexto `node:vm`, pede cada ícone a `iconCanvas`, calcula o hash SHA-1 do RGBA (cor + forma) e do canal alfa isolado (só a silhueta) e acusa qualquer par igual. Também conta os pixels pintados, para pegar ícone vazio ou quase invisível. Com `--sheet pasta/` grava cinco folhas de contato ampliadas para conferência a olho.

- `npm run test:art` → imprime por grupo e termina com `ARTE OK — nenhuma peça repete outra` (saída 0) ou a lista dos pares repetidos (saída 1).
- Antes disso ele lista, por tipo de acessório, quantos desenhos distintos existem e acusa se duas peças do mesmo tipo caíram na mesma variante.
- `node dev/test_gear_art.js --sheet /tmp/folhas` → grava `minerios.png`, `barras.png`, `armaduras.png`, `acessorios.png` e `picaretas.png`.
- Precisa de `@napi-rs/canvas`: `npm --prefix dev/pdf install` instala a mesma biblioteca que gera este PDF.

USA fs, vm, crypto, @napi-rs/canvas
USADO POR npm run test:art

dev/test_swords_x.js

TESTE 134 linhas

As 80 espadas novas, sem navegador

Executa **todos** os ganchos de efeito de cada uma das 80 espadas novas em um motor falso que grava chamadas — e falha em qualquer exceção.

Em vez de abrir um navegador, substitui o motor real (dano, projéteis, explosões, feixes, meteoros, buffs) por espiões que só anotam o que foi chamado. Depois dispara `onSwing`, `onApex`, `onHit`, `onProjHit`, `onKill`, `tick` e `stats` de cada espada. Resultado atual: 80 espadas, 0 problemas.

USA fs, vm, stubs de canvas
USADO POR verificação dos tiers 7–10

Testes de navegador (Playwright)

Os arquivos `test_*.js` abrem o `index.html` em um Chromium headless e dirigem o jogo pelos mesmos ganchos que o jogador usaria. Cada um cobre uma área e falha com código de saída 1 se encontrar exceção, erro de console ou resultado inesperado.

Arquivo	O que verifica
test_worlds.js	Mundos: criação sem semente, sementes inéditas, miniaturas, renomear/excluir, persistência ao recarregar, fidelidade bloco a bloco, autosave, migração e cota.

Arquivo	O que verifica
test_device.js	Exportar/importar entre “aparelhos” (contextos isolados), arquivos adulterados, celular, Firefox, sem IndexedDB.
test_gamepad.js	Controle simulado (dev/gamepad_mock.js): mirar, atacar, andar nos painéis.
test_touch.js	Direcional, ação com arrasto, pulo, painéis, zoom, cura e pausa em tela de toque.
test_input.js	Teclado e mouse reais: andar, pular, minerar, colocar bloco, crafting, catálogo e mapa.
test_arsenal_clicks.js	Clica em (quase) tudo do Arsenal: todas as abas de tier, itens, inimigos e mundo.
test_enemies.js	Todos os inimigos e variantes elementais contra o jogador, sem exceções.
test_bosses.js	Sprites e fases dos 4 chefes clássicos, íris que segue o jogador, tinta de status, cache e caixas de colisão.
test_bx.js	Cada um dos 50 chefes novos: invocação, 3 fases, todos os golpes, morte, loot e limpeza de lacaios.
test_bx_art.js	Arte dos 50 chefes: silhueta própria, corpo escuro com brilho, paleta curta, colisão dentro do desenho.
test_progress.js	Morte e renascimento, derrota dos 4 chefes, drops, bandeiras, estágio e minérios de Hardmode.
test_npc_save.js	Salas válidas, spawn de NPC, painel da loja e ida e volta do save pelo IndexedDB.
test_fluids.js	Água + lava = obsidiana, areia caindo e plantas sem apoio.
test_sandbox.js	O jogo dentro de um iframe sandbox (sem localStorage): o salvamento falha com aviso, não com exceção.
test_skin.js	Interface única e copas das árvores (silhueta, contorno, lote de desenho).
test_swords.js	As 120 primeiras espadas em combate: cada uma ataca por segundos e precisa causar dano.
soak.js	Estresse: ~45.000 passos de simulação em 14 cenários (todas as camadas e estágios) procurando travamentos e vazamentos.
smoke.js	Abre o jogo, tira a captura do título e mostra o que saiu no console.
http_check.js	Ponta a ponta por http: carrega, salva, recarrega e continua — o teste do fluxo real de jogador.

Ferramentas

`dev/lib.js`

FERRAMENTA 91 linhas

Base compartilhada dos testes de navegador

Abre o Chromium, cria mundos por semente determinística, coleta erros de página e monta folhas de contato a partir de capturas.

Centraliza o que todos os testes precisam: lançar o navegador com as opções certas, esperar o jogo entrar em `play`, forçar a semente do próximo mundo (`window.__nextSeed`) e juntar capturas em uma única imagem para conferência visual.

launch · startWorld · waitPlay · collectErrors · makeSheet · run

USA playwright
USADO POR todos os test_*.js

dev/art_sheet.js

FERRAMENTA 81 linhas

Folha de contato das 80 espadas novas

Renderiza os sprites reais das espadas 121–200 com `@napi-rs/canvas`, em uma grade 20×4 com a cor do tier em cada célula.

Serve para conferir a arte sem abrir o jogo: cada célula mostra o sprite no tamanho real do mundo, com uma faixa da cor do tier e o número. É a ferramenta que guiou o ajuste fino de pontas, guardas e halos durante a criação dos tiers 7–10.

- `node dev/art_sheet.js` grava `espadas_t7_10.png` na pasta atual.
- O segundo argumento é o caminho de saída; o terceiro é a escala.

USA @napi-rs/canvas (opcional)
USADO POR conferência visual da arte

dev/gamepad_mock.js · dev/host.html

FERRAMENTA 17 linhas

Simulador de controle e página hospedeira

Injeta uma Gamepad API falsa (ou bloqueada) antes do jogo carregar; e hospeda o jogo em um iframe `sandbox` para testar ambientes restritos.

USA —
USADO POR test_gamepad.js · test_sandbox.js

14. Como rodar as verificações

A bateria curta não precisa de nada instalado além do Node. A bateria de navegador precisa do Playwright e do Chromium baixado uma vez.

Comando	O que roda
npm run build	Gera o index.html a partir de src/.
npm run serve	Publica o jogo em http://localhost:8080/.
npm test	Validação rápida: catálogo, alcance das receitas, sementes, layouts e arquivo de mundo.
npm run test:browser	A bateria completa de navegador (precisa de `npm install` e `npx playwright install chromium`).
node dev/test_swords_x.js	As 80 espadas novas, com motor falso, em cerca de 1 segundo.
node dev/art_sheet.js	Folha de contato da arte das espadas novas.

Fluxo recomendado depois de mexer no código

1) `node build.js` · 2) `node dev/validate.js` · 3) `node dev/test_swords_x.js` · 4) `npm test` · 5) abrir no navegador e jogar. Se a mudança for visual, gere a folha de contato antes.

PARTE V

A pasta docs/

O catálogo gerado, a lista de chefes e as capturas usadas na documentação.

15. Catálogo, chefes e imagens

docs/catalogo_200_espadas.md / .csv

GERADO 295 linhas

Catálogo das espadas em texto

Tabela com as 200 espadas, tier por tier: nome, dano, velocidade, knockback, crítico, mecânica, receita e estação — gerada a partir dos dados do jogo.

Serve para consultar e comparar espadas sem abrir o jogo, e para planilha (o CSV usa ponto e vírgula e BOM, abrindo direto no Excel em português). Cobre os 10 tiers, inclusive os 7–10, que vivem em `11b_swords_x.js`.

- Recriado com `CATALOGO=1 node dev/validate.js`.
- O CSV tem uma linha por espada; o MD agrupa por tier com a regra de cada um.

USA 11_swords.js
USADO POR README, consulta

docs/catalogo_equipamentos.md

GERADO 361 linhas

Catálogo do equipamento

As 10 famílias, com os 50 minérios (tier e picareta exigida), as 50 armaduras (defesa e passiva) e os 50 acessórios (de qual chefe vêm e o que fazem).

Gerado pelo mesmo `CATALOGO=1 node dev/validate.js`, a partir de `11c_gear_x.js` e `10b_gear_fx.js`. Serve para comparar peças e para saber de qual chefe sai cada acessório sem abrir o jogo.

USA 11c_gear_x, 10b_gear_fx
USADO POR README, consulta

docs/chefes_novos.md

GERADO 83 linhas

Lista dos 50 chefes

Nome, ato, atributos, mecânica única, acessório exclusivo, condição de invocação e recompensas dos 50 chefes novos.

É gerado pelo `validate.js` a partir dos dados de `13d..13g_bx_*.js`, então nunca fica desatualizado em relação ao jogo.

USA 13c_bossx, 13d..13g
USADO POR README, consulta

docs/img/*.png

IMAGENS 13 linhas

Capturas de tela

Treze imagens (menu, combate, biomas, arco de ataque, efeitos, interface, tiles, mapa, controles, chefes) usadas no README.

São capturas reais do jogo, feitas pelos testes de navegador, e servem como referência visual do que cada sistema produz na tela.

USA —
USADO POR README.md

PARTE VI

Como funciona por dentro

Seis capítulos transversais: o caminho de um golpe, o formato de uma espada, os dois pipelines de arte, o salvamento, os tiers novos e as armadilhas do código.

16. O caminho de um golpe

Seguir um único clique do mouse até o número de dano na tela é a melhor maneira de entender como os módulos se encaixam. O percurso passa por sete arquivos diferentes:

1

Entrada

`mousedown` no canvas marca `G.mouse.down[0]` (16_main).

2

Jogador

`updatePlayer` vê o botão pressionado, checa o tempo de recarga e chama `startSwing` (12_player).

3

Arco

`startSwing` cria a instância do golpe, dispara `sfx("swing")` e o gancho `onSwing` da espada (09b_swing → 10_fx / 11b).

4

Ápice

No meio do arco, `updateSwings` marca `apex` e chama `onApex`: é aqui que os projéteis, ecos e domínios nascem, já apontados para o cursor.

5

Colisão

`swingHit` testa o segmento da lâmina contra a caixa de cada inimigo; cada instância guarda quem já atingiu.

6

Dano

`hitEnemy` aplica crítico, defesa, elemento e status, dispara `onHit` e devolve o resultado (09_combat).

7

Consequências

Número flutuante, partículas, som, “hit stop”, drops; se a vida zerar, `killEnemy` distribui loot e bandeiras de progresso.

Os ganchos, em ordem

Os ganchos, em ordem

Gancho	Quando roda	Para que serve
stats	ao equipar / recalcular	Mudar atributos do jogador (dano, crítico, velocidade, vida).
init	ao equipar	Preparar estado interno do efeito.
swing	ao iniciar o golpe	Alterar o próprio arco (alcance, duração, lado).
onSwing	ao iniciar o golpe	Efeitos de largada (som, brilho, buff).
mod	durante o golpe	Multiplicador de velocidade de ataque.
onApex	no meio do arco	Criar projéteis, ecos, domínios, meteoros.
onHit	ao acertar inimigo	Explosões, marcas, roubo de vida, acúmulos.
onProjHit	projétil acerta	Reações de projétil (fragmentar, explodir, ricochetear).
onKill	ao matar	Almas, explosões, cura, invocação.
tick	todo quadro	Efeitos contínuos: auras, acumuladores, temporizadores.

17. Como uma espada é definida

Toda espada é um objeto de dados. As 120 primeiras são criadas pela fábrica `S(...)` em `11_swords.js`; as 80 novas usam `S2(...)` e uma receita de arte `A2(...)` em `11b_swords_x.js`. A assinatura é a mesma ideia: número, nome, dano, velocidade, knockback, aparência, efeitos, texto e receita.

```
S2(121, 'Nome', dano, velocidade, kb, A2('familia', {...}), [FX2.eco(...)], 'mecânica', 'ingrediente:qtd,...')
```

Campo	Significado
num / id	Número da espada (1–200) e id `sNNN`; o tier é `ceil(num/20)`.
baseDamage / attackSpeed	Dano por golpe e golpes por segundo.
knockbackForce	Empurrão aplicado no inimigo.
critChance	Crítico; se omitido, usa `DEF_CRIT[tier]` (tiers 7–10: 12%, 15%, 18%, 22%).
look	Aparência: formato, cor, brilho e — nas novas — a receita `art2` de pintura.
passiveEffects	Lista de efeitos da biblioteca (`FX` ou `FX2`) combinados.
txt	Descrição da mecânica única, exibida no tooltip e no catálogo.
station / receita	Onde é criada e com quais ingredientes (`TIER_STATION` define o padrão).

> **Cuidado ao criar uma espada nova**
>
> A receita precisa ter ingredientes que existam em `IT` e uma estação válida, o nome não pode repetir, o dano deve cair na faixa do tier (o `validate.js` cobra isso) e o número tem de estar livre. Depois de acrescentar, rode `node dev/validate.js` e `node dev/test_swords_x.js`.

18. Como a arte é gerada

Não existe um único arquivo de imagem no jogo: **tudo é desenhado por código**, em três pipelines diferentes.

Pipeline	O que usa	Como funciona
Vetorial + pixelização	Espadas 1–120, ícones de itens, bonecos, tiles	Desenha formas suaves num canvas pequeno, binariza o alfa, adiciona contorno escuro e sombreamento de cima-esquerda.
Pintura pixel a pixel	Espadas 121–200 e chefes	Um buffer RGBA recebe pixels um a um, com rampas de tom, bisel, contorno e pixels emissivos — sem nenhuma suavização.
Formas por semente	Minérios, barras, armaduras, acessórios, picaretas	A semente do item sorteia o corte, o comprimento e os enfeites; o brasão da família entra por cima.

O resultado é sempre **cacheado**: um sprite é gerado uma vez e reaproveitado. Os ícones da interface são gerados em 32×32, centralizando a espada em diagonal e usando a caixa real do desenho (`sw`/`sh`) para que nenhuma espada longa seja cortada. No mundo, o sprite é desenhado em tamanho real, com ampliação por fator inteiro — daí o pixel quadrado.

Regras de estilo que o código respeita

- Uma única fonte de luz (cima-esquerda): gume claro em cima, dorso escuro embaixo.
- Poucos tons por material, separados por tons escuros em vez de linhas.
- Contorno escuro de 1 px em toda a silhueta: é o que faz o objeto “descolar” do fundo.
- Brilho por cor, com núcleo quase branco e halo semitransparente pontilhado.

19. Salvamento, mundos e arquivos

O jogo guarda **vários mundos**, não um único save. Cada mundo tem metadados leves (para listar rápido) e um registro pesado com os blocos comprimidos. Tudo vive no IndexedDB do navegador, na base `espadas120`.

Loja	Conteúdo
worlds	Cartão do mundo: nome, semente, modo, tempo jogado, dia, estágio, chefes derrotados, miniatura, aparelho de origem.
data	O mundo em si: blocos comprimidos em RLE + base64, jogador, inventário, baús, NPCs, bandeiras.
seeds	Todas as sementes já sorteadas — nenhuma se repete, nem depois de excluir o mundo.

O salvamento é feito em **fila**: nunca dois ao mesmo tempo. O autosave roda a cada 60 s; salvar manualmente também refaz a miniatura. Se o navegador não tiver IndexedDB (iframe restrito, alguns modos privados), o jogo avisa e mantém os mundos apenas em memória, durante a sessão.

Exportar e importar

Cada mundo pode virar um arquivo `.json` autocontido, com CRC-32 para detectar corrupção. Na importação, **nada é confiado**: o CRC é verificado, campos desconhecidos são descartados, valores são presos à faixa válida, itens inexistentes são removidos e as camadas precisam ter o tamanho certo. Um arquivo ruim nunca quebra o jogo — no pior caso, é recusado com uma mensagem.

20. Os Tiers 7–10 (espadas 121–200)

A adição mais recente: **quatro tiers novos, 20 espadas cada, 80 no total**, levando o catálogo de 120 para 200. Nome, atributos, mecânica e desenho são exclusivos, e a arte é pintada pixel a pixel — não é variação das 120 antigas.

Tier	Nome	Cor	Espadas	Dano	Crítico	Estação
7	Forja Estelar	#eab308	121–140	300–360	12%	Bigorna Avançada
8	Relicário do Vazio	#db2777	141–160	470–620	15%	Bigorna Avançada
9	Eco Primordial	#06b6d4	161–180	690–880	18%	Forja Celeste
10	Origem	#f8fafc	181–200	980–1400	22%	Forja Celeste

Os 25 efeitos novos (FX2)

Os efeitos novos ficaram em um objeto próprio (FX2), sem tocar na biblioteca antiga: eco do golpe, leque de lâminas, nova, cometa, varredura, corrente, marca que detona, domínio, gravidade, escudo, dobra, divisão, almas perseguidoras, execução, explosão de sangue, convergência, ciclo, fenda, espelho, tempestade, parada no tempo, linha de condenação, orbes, singularidade e big bang.

O que foi verificado

- 200 espadas, 20 por tier, nomes únicos, dano dentro da faixa e receitas válidas (`validate.js`).
- 200/200 espadas craftáveis, nenhum ingrediente sem fonte (`reach.js`).
- Os oito ganchos de cada uma das 80 espadas executam sem exceção (`test_swords_x.js`).
- Catálogo com abas T1–T10 + “Todas”, 20 slots por aba e 200 em “Todas”; 379 ícones, tooltips, venda, save e painéis sem erro.

> **Correções que vieram junto**
>
> Ícone que não era espada quebrava o painel (`ReferenceError`); o catálogo escondia T8–T10 e a aba “Todas”; o preço de venda das espadas novas era indefinido; eco e espelho geravam ecos de ecos (acertos exponenciais, travando o jogo); e a lista de projéteis estourava no teto de 700. Tudo corrigido e coberto por teste.

21. Convenções e armadilhas do código

Convenções

- Cada arquivo começa com um comentário-cabeçalho dizendo o que ele faz — é a melhor documentação do projeto.
- O prefixo numérico do arquivo é a ordem de carga; ao criar um módulo, escolha o número entre os vizinhos lógicos (por isso existem `06b`, `09b`, `13d`…).
- Constantes de conteúdo ficam nos módulos de dados; nada de números mágicos espalhados pelo motor.
- Funções que tocam o DOM só são chamadas em tempo de jogo: os módulos carregáveis pelo validador não executam nada no topo.

Armadilhas

Armadilhas

Armadilha	Por que acontece	Como evitar
Ordem de carga	Tudo compartilha um escopo; `const` no topo de um arquivo não existe antes de sua linha rodar.	Use o prefixo numérico correto e declare dependências depois de suas bases.
Editar o index.html	O build reescreve o arquivo inteiro.	Edite `src/` e rode `node build.js`.
Variável implícita	Um nome digitado errado dentro de uma função vira erro só naquele caminho (foi o caso do ícone que usava `it`).	Rode os testes: eles exercitam inventário, arsenal e todos os itens.
Recursão de efeitos	Eco que gera eco dobra os acertos a cada geração.	Efeitos disparados por golpes “fantasma” precisam checar `inst.ghost`.
Listas com teto	`spawnProj` descarta os 50 mais antigos quando passa de 700 — o laço pode encontrar buracos.	Ao iterar, tolere elementos ausentes.
Preços e tabelas por tier	Arrays indexados por tier silenciosamente devolvem `undefined` acima do último índice.	Estenda a tabela e use valor de reserva.

A. Todos os módulos em uma tabela

Referência rápida: nome, linhas e papel de cada um dos 46 módulos, na ordem de carga.

Módulo	Linhas	Papel
00_util.js	84	Matemática, RNG determinístico, Perlin, cores
01_data.js	259	Tiers, tiles, paredes, itens, receitas
02_world.js	230	Matriz de blocos, líquidos, quedas, árvores
03_worldgen.js	439	Gerador procedural em fatias
04_light.js	106	Iluminação dinâmica por opacidade
05_gfx.js	346	Sprites 16×16 dos tiles e auto-tiling
05b_pixel.js	156	Pixelização e bonecos humanos
05c_canopy.js	164	Copas das árvores por vizinhança
05d_bosspaint.js	151	Ferramentas de pintura dos chefes
05e_bossart.js	293	Sprites dos 4 chefes clássicos
05f_bxart.js	68	Infraestrutura da arte dos 50 chefes
05g_bxart1.js	142	Arte dos chefes do Ato I (8)
05h_bxart2.js	243	Arte dos chefes do Ato II (14)
05i_bxart3.js	236	Arte dos chefes do Ato III (16)
05j_bxart4.js	179	Arte dos chefes do Ato IV (12)
06_icons.js	243	Ícones de itens e sprites das espadas 1–120
06b_swordart.js	649	Arte pixel a pixel das espadas 121–200
06c_gearart.js	760	Arte própria de cada peça de equipamento
07_sky.js	100	Céu, astros, nuvens e colinas
08_phys.js	145	Estado global G, colisão, partículas, drops
08b_audio.js	60	Efeitos sonoros sintetizados
09_combat.js	334	Dano, status, projéteis, explosões
09b_swing.js	95	Arco do golpe, ápice, colisão da lâmina
10_fx.js	276	Biblioteca de 101 efeitos de espada
11_swords.js	157	Catálogo das 120 primeiras espadas
11b_swords_x.js	425	Tiers 7–10: 80 espadas + 25 efeitos
10b_gear_fx.js	498	Mecânicas de acessórios, armaduras e conjuntos
11c_gear_x.js	256	50 minérios, 50 armaduras, 50 acessórios, 5 picaretas
12_player.js	360	Jogador, inventário, criação, mineração
13_enemies.js	334	Inimigos, IA, spawn e drops
13b_bosses.js	78	Os 4 chefes clássicos
13c_bossx.js	306	Motor dos 50 chefes novos

Módulo	Linhas	Papel
13d_bx_a1.js	120	Chefes novos do Ato I (8)
13e_bx_a2.js	269	Chefes novos do Ato II (14)
13f_bx_a3.js	263	Chefes novos do Ato III (16)
13g_bx_a4.js	187	Chefes novos do Ato IV (12)
14_draw_enemies.js	236	Desenho de inimigos e chefes
14b_draw_bx.js	36	Desenho dos chefes novos
15_ui.js	319	HUD, inventário, catálogo, mapa, ajuda
15b_npc.js	145	Moradia, NPCs e lojas
15c_save.js	241	Biblioteca de mundos (IndexedDB, RLE)
15d_gamepad.js	204	Suporte a controle
15e_worlds.js	179	Menu do título e “Meus Mundos”
15f_transfer.js	261	Aparelho, proteção e arquivo de mundo
15g_touch.js	80	Controles de toque
16_main.js	270	Loop, render, input, câmera

Total: **46 módulos · 10.757 linhas** de JavaScript, mais 404 linhas de CSS e template.

B. Comandos e ganchos de teste

Comando	Efeito
node build.js	Compila src/ em index.html.
node serve.js [porta]	Serve o jogo por http (padrão 8080).
node dev/validate.js	Valida catálogo, receitas e chefes.
CATALOGO=1 node dev/validate.js	Idem, e reescreve o catálogo em docs/.
node dev/reach.js	Confere se todo item é obtível.
node dev/test_swords_x.js	Executa os ganchos das 80 espadas novas.
npm test / npm run test:browser	Baterias de verificação.

Ganchos expostos no navegador

O `16_main.js` publica alguns atalhos em `window` para que os testes dirijam o jogo sem depender de cliques: `__G` (estado), `__step(dt)`, `__render()`, `__start(opções)`, `__SW` (catálogo), `__IT` (itens), `__Worlds`, `__chooseSeed`, `__PAD` e `__padUpdate`. Eles não afetam o jogador — são a interface de automação do projeto.

Resumo final

O 101spas é um jogo de navegador em arquivo único, construído por concatenação de 46 módulos globais. Os dados de conteúdo estão separados do motor; o desenho é gerado por código em dois pipelines (vetorial pixelizado e pintura pixel a pixel); o estado vive em G; a interface é DOM sobre um canvas de baixa resolução ampliado por fator inteiro; e a verificação é feita em duas camadas — Node para dados e lógica, Chromium para o jogo de verdade.